

Cartograph

Natural-Language Querying of Complex Data Warehouses with Schema-Graph Retrieval and Explainable Join-Path Selection

Project description · Jagath Manjunath · September 2026

The situation

A mid-sized company keeps its business history in a data warehouse — sales, customers, products, stores, deliveries, returns. Forty tables, all connected to each other by keys.

The people who need answers are managers and analysts. They know exactly what they want to ask: *"which product category made the most revenue in the western region last quarter?"* What they don't know is SQL, or which of forty tables holds what, or how those tables join together.

So they raise a ticket with the data team and wait three days. The data team becomes a bottleneck, and the company asks fewer questions than it should.

Why "just use an LLM" doesn't solve it

LLMs write SQL fluently. Point one at a small database and it works.

Point it at a real warehouse and it starts failing — not on syntax, but on **grounding**. Give a model forty tables and three hundred columns and ask about "revenue in the western region," and it must work out which of six columns containing the word *region* is meant, and which chain of joins connects a sale to a region.

The failure that matters

In a retail warehouse there are usually two valid paths from a sale to a region:

- through the **store** where the sale happened
- through the **customer's delivery address**

Both are real foreign-key paths. Both are structurally correct. For an online retailer they give materially different answers.

The model picks one. The query runs. It returns a clean, well-formatted table of categories and revenue figures. **And there is no error.** Nothing indicates which interpretation was taken.

The manager reads a confident number and decides on it, with no way to know it answered a different question than the one asked. That is a **silent** failure — worse than a loud one, because it is not merely wrong, it is wrong and hidden.

What Cartograph does about it

First — stop showing the model the whole schema

A database already declares its own structure: tables, columns, primary keys, foreign keys. Read that metadata and build a **graph** — tables and columns are nodes, key relationships are edges. No AI involved; it is reading what is

already there.

When a question arrives, find the few schema pieces that look relevant, then **walk the graph** to find the join paths that actually connect them. Hand the model a small, relevant slice — the tables it needs and the roads between them — instead of a forty-table dump it must guess its way through.

Less to get wrong.

Second — make the reasoning visible

Every answer comes back with a **trace**: which tables and columns were retrieved, which join path was chosen, what other paths existed, and a plain-English account of how the query was assembled.

So when there are two ways to reach "region," the user *sees* that, and sees which one was taken. Someone with domain knowledge can say "no, I meant the other one" — before acting on the number.

The thesis: reduce the errors, and expose the ones that remain.

One question, end to end

"Which product category generated the highest revenue in the western region last quarter?"

1. The system finds the schema pieces that look relevant — sales, products, categories, something region-shaped
2. It walks the graph and discovers **two** routes to region, and records that ambiguity rather than hiding it
3. It hands the model a small subgraph — the relevant tables plus the join paths
4. The model writes SQL
5. Before anything runs, the SQL is parsed and checked: does it reference real columns, is it read-only, is it safe
6. It executes on a read-only connection with a timeout
7. Back comes the answer table, **plus** the trace: what was retrieved, the path taken, the path not taken, and why

So what I am building

A web app. You type a question in English, you get a table of results, and beside it a panel showing exactly how the system got there — with the schema graph drawn out and the route it took highlighted.

Underneath: a Python service that reads database structure, retrieves intelligently over a graph, talks to an LLM, validates the SQL before it runs, and records every decision it made.

Everything else — the security hardening, the evaluation, the deployment — is making that idea trustworthy enough to actually let someone use.